

Supervised Learning

A Review

DS 4440 · LECTURE 1

1. The setup

We are given **training data** $(\mathbf{X}, \mathbf{y})$. The inputs are a collection of N examples, each described by D features:

$$\mathbf{X} = \{\mathbf{x}_1, \dots, \mathbf{x}_N\} = \begin{bmatrix} x_{11} & \cdots & x_{1D} \\ \vdots & \ddots & \vdots \\ x_{N1} & \cdots & x_{ND} \end{bmatrix}, \quad \mathbf{X} \in \mathbb{R}^{N \times D},$$

where row i is the feature vector $\mathbf{x}_i \in \mathbb{R}^D$ of the i -th example, and column j is feature j measured across all examples. Each example carries a label:

$$\mathbf{y} = [y_1 \ \cdots \ y_N], \quad \mathbf{y} \in \mathbb{R}^N.$$

The goal. Find **parameters** θ for a function f_θ such that

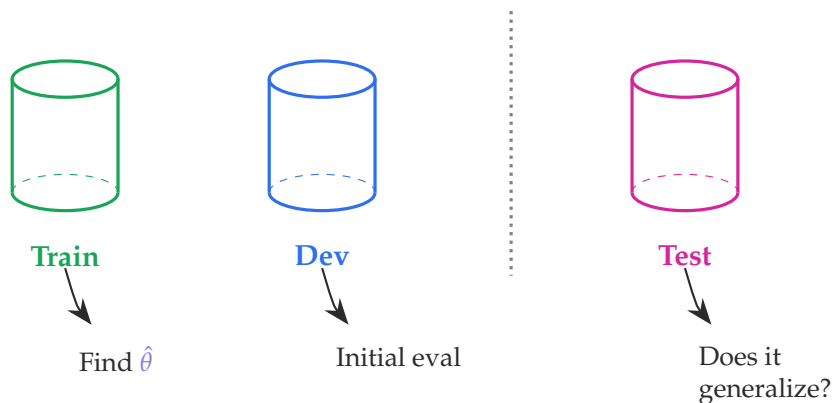
$$f_\theta(\mathbf{X}) = \mathbf{y},$$

and — critically — such that this **generalizes** beyond $\mathbf{X}$.

Fitting the training data is easy — a lookup table does it perfectly. What we want is a function that behaves well on examples it has never seen.

2. Train, dev, test

To keep ourselves honest, the data is partitioned into three pools with three different jobs.



Train.

The only split the learning algorithm is allowed to look at. We search over θ here.

Dev.

Our initial evaluation — model selection, hyperparameters, sanity checks. It is peeked at repeatedly, so performance on it is mildly optimistic.

Test.

Untouched until the very end: the only honest estimate of generalization, and it stops being one the moment we tune against it.

3. What “learning” means

Learning means finding a $\hat{\theta}$ such that the **loss** $\mathcal{L}$ on $(\mathbf{X}, \mathbf{y})$ is minimized. The loss is a function that scores how wrong a prediction is; minimizing it over the data is the whole procedure.

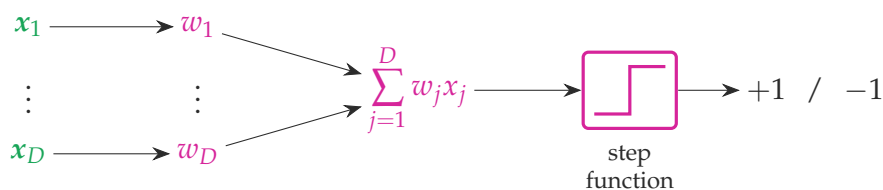
$$\hat{\theta} = \arg \min_{\theta} \sum_{i=1}^N \mathcal{L}(\underbrace{f_{\theta}(\mathbf{x}_i)}_{\hat{y}_i}, y_i)$$

This is machine learning.

Everything else — architectures, optimizers, tricks — is a choice of f_{θ} , a choice of $\mathcal{L}$, or a way of carrying out the arg min.

4. One choice: the perceptron

Take f_{θ} to be a weighted sum of the features passed through a step function. The parameters are the weights, $\theta = \mathbf{w}$.



Concretely, the prediction for example i is

$$\hat{y}_i = \begin{cases} 1 & \text{if } \mathbf{w}\mathbf{x}_i > \tau, \\ -1 & \text{otherwise,} \end{cases}$$

and the loss is **0/1 loss** — did we get it right or not?

$$\mathcal{L}(\hat{y}_i, y_i) = \begin{cases} 0 & \text{if } \hat{y}_i = y_i, \\ 1 & \text{otherwise.} \end{cases}$$

Learning the weights

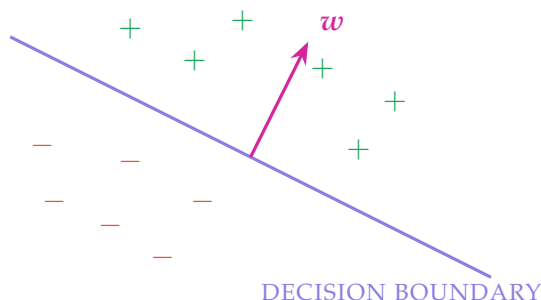
Algorithm 1: Perceptron learning ($\theta = \mathbf{w}$)

```
 $\mathbf{w} \leftarrow \mathbf{0}$ 
for  $(\mathbf{x}_i, y_i) \in (\mathbf{X}, \mathbf{y})$  do
    if  $\text{step}(\mathbf{w}\mathbf{x}_i > \tau) \neq y_i$  then
         $\mathbf{w} \leftarrow \mathbf{w} + y_i \mathbf{x}_i$            // correct toward this example
    end
end
return  $\mathbf{w}$ 
```

★ This assumes the **bias** is baked into $\mathbf{x}_i$ and $\mathbf{w}$: append a constant 1 feature to every $\mathbf{x}_i$ and let its weight absorb the threshold. (See the Colab notebook.)

5. Why does this work?

The update rule $\mathbf{w} \leftarrow \mathbf{w} + y_i \mathbf{x}_i$ looks almost too simple. Here is the geometry behind it.



The **weight vector is perpendicular to the decision boundary**. To see why, consider the set of points where the score is exactly zero:

$$\mathbf{w}\mathbf{x}_i = 0.$$

That is exactly the set of $\mathbf{x}_i$ orthogonal to $\mathbf{w}$: a hyperplane through the origin with $\mathbf{w}$ as its normal. Points on the $\mathbf{w}$ side score positive and are labeled +1; the rest score negative and are labeled -1. Changing $\mathbf{w}$ rotates the hyperplane.

The update always helps that example

Suppose we make an **incorrect** prediction for an $\mathbf{x}_i$ that should be +. Since $y_i = +1$, the update is

$$\hat{\mathbf{w}}' \leftarrow \hat{\mathbf{w}} + \mathbf{x}_i.$$

The new score assigned to that same example is

$$\hat{\mathbf{w}}' \cdot \mathbf{x}_i = (\hat{\mathbf{w}} + \mathbf{x}_i) \cdot \mathbf{x}_i = \hat{\mathbf{w}} \cdot \mathbf{x}_i + \underbrace{\|\mathbf{x}_i\|^2}_{\text{positive}}.$$

The score for the misclassified example moves *up* by $\|\mathbf{x}_i\|^2 > 0$, i.e. toward the +1 side of the boundary. Symmetrically, for a misclassified negative example ($y_i = -1$) the update subtracts $\mathbf{x}_i$ and the score moves down by the same amount. Every update nudges the boundary in the direction that fixes the mistake it just made.

The guarantee covers only the offending example; the same update may break others. But if the data is linearly separable, repeated sweeps converge to a separating $\mathbf{w}$ in finitely many updates.